

Learning to Place Guards by Reinforcement: A Geo-Free Neural Policy for the Vertex-Guard Art Gallery Problem

Domagoj Ševerdija Jurica Maltar Nathan Chappel
Domagoj Matijević

Abstract

We study whether a neural policy can learn to place vertex guards on a simple polygon from a coverage-aware reward over its own rollouts, with no geometric oracle at inference. The method is an LSTM pointer-network policy trained with preference optimization on Bradley–Terry pairs of self-generated rollouts, decoded greedily from vertex coordinates alone at test time — no visibility matrix, no per-vertex area feature, no oracle call. On held-out polygons the policy is competitive in guard cardinality but leaves a residual tail of under-covered polygons, and the tail grows on substantially larger out-of-distribution polygons. To separate representation quality from decoder quality, we train a small single-shot per-vertex classifier over the frozen encoder embeddings and the policy’s own seed indicator, with a scalar threshold as its operating point; the classifier closes the feasibility tail, both in distribution and out-of-distribution, again with no geometric input at inference. We read the combined result as a bounded measurement of what is learnable in this geo-free regime, and as an empirical separation of representation from decoding. We do not claim to compete with classical greedy or local-search heuristics on guard count. Vertex-guard optima used in the $|S|/\text{OPT}$ reporting are computed by us; the public instance library we draw polygons from provides point-guard solutions, which are not the vertex-guard optimum.

1 Introduction

The Art Gallery Problem (AGP) asks for the minimum number of guards needed to observe every point of a simple polygon [10, 12], with applications in surveillance, sensor placement, and robotics [6, 3]. We focus on the *vertex-guard* variant, in which guards must be chosen from the polygon’s vertices; the problem remains NP-hard and is a discrete instance of geometric set cover with non-uniform, polygon-dependent visibility regions. The question this paper investigates is whether a neural policy can learn to place such guards from coordinates alone, trained from a coverage-aware reward signal over its own rollouts, with no visibility matrix and no geometric oracle at the moment it chooses guards. We

are not asking whether a learned solver can match a classical greedy heuristic on guard count — it cannot, and we do not claim it does. We are asking whether the underlying combinatorial-geometric task is learnable when the model is denied any explicit geometric input at test time.

The reinforcement method we evaluate is an LSTM pointer-network policy trained with Preference Optimization (PO) [13] on Bradley–Terry preference pairs [4] drawn from the policy’s own rollouts. The policy emits a vertex-index sequence with an EOS token; the reward couples coverage and guard usage. PO replaces REINFORCE advantages [17, 1] with binary preferences between pairs of rollouts, sidestepping REINFORCE’s variance and the brittleness of supervised cloning of search labels. We refer to inference under this policy as *geo-free*: at test time the network sees only the polygon’s vertex coordinates, with no precomputed visibility matrix, no CGAL output, and no per-vertex area feature.

The policy works, to a degree. On a held-out in-distribution split it reaches mean coverage of roughly 0.97 at $|S|/\text{OPT} \approx 1.09$, which is competitive in cardinality with classical baselines on these polygons. The honest limit of the result is the tail of the per-polygon coverage distribution: a non-trivial fraction of polygons end up below the feasibility line that one would want any deployed solver to clear, and the failure rate grows on out-of-distribution polygons whose vertex counts exceed the training range. The reinforcement method, by itself, is not feasibility-guaranteed.

To check whether this residual reflects a true limit of the reinforcement-learned representation or only a calibration problem in the decoder, we train a small per-vertex classifier that reads only the frozen encoder embeddings of the policy and predicts a vertex-inclusion probability, controlled by a single scalar threshold. The classifier has no access to visibility information at inference. It closes the feasibility tail: every in-distribution held-out polygon clears the coverage gate, and out-of-distribution failures fall by approximately two orders of magnitude. We read this as direct evidence that the reinforcement-trained encoder already carries enough geometric structure to support a feasibility-guaranteeing decision, and that the policy’s coverage tail was a decoder-calibration limit rather than a representation limit. The cost of recovering feasibility is a controlled rise in guard count, which we report explicitly.

Our contribution is twofold. We measure what a reinforcement learner can and cannot extract from a coverage-aware reward on vertex-guard AGP when denied geometric input at inference, and we use a small probe over the frozen encoder to separate the question of representation quality from the question of decoder quality. We do not claim to beat classical greedy or local-search heuristics on guard count; we claim, and document, that vertex-guard placement is learnable to a measurable degree by a reinforcement method that never sees an explicit visibility object.

The remainder of the paper is organized as follows. Section 2 fixes notation and the geo-free constraint. Section 3 reviews related work. Section 4 describes the reinforcement method, what it does and does not absorb, and the repre-

sentation probe. Section 5 describes the experimental setup. Section 6 reports the results. Section 7 reads the results against the question. Section 8 states limitations and Section 9 concludes.

2 Problem and Notation

2.1 Polygons and visibility

Let $s \subset \mathbb{R}^2$ denote a simple polygonal region (the closed set consisting of its interior and boundary), with n vertices at coordinates $x_1, \dots, x_n \in \mathbb{R}^2$ and vertex index set $V(s) = \{1, \dots, n\}$. A *guard* is any point $g \in s$ with unobstructed 360° visibility inside s . For a finite guard set $G \subseteq s$, the visibility region and area-normalized coverage are

$$\text{Vis}(G) = \{q \in s : \exists g \in G, \overline{gq} \subseteq s\}, \quad \text{Cov}(G \mid s) = \frac{\text{Area}(\text{Vis}(G))}{\text{Area}(s)} \in [0, 1]. \quad (1)$$

We write $\text{Cov}(G)$ throughout when s is clear from context.

2.2 The vertex-guard variant

In the *vertex-guard Art Gallery Problem (AGP)* guards are restricted to the polygon’s vertices: $S \subseteq V(s)$, with physical positions $\{x_i : i \in S\}$. The coverage of S is $\text{Cov}(S)$, with visibility region $\text{Vis}(S) = \{q \in s : \exists i \in S, \overline{x_i q} \subseteq s\}$. We define the *vertex-guard optimum* directly:

$$\text{OPT}(s) = \min\{|S| : S \subseteq V(s), \text{Cov}(S) = 1\}. \quad (2)$$

Restricting guards to $V(s)$ reduces placement to a finite discrete choice over n vertices — the natural action space for a pointer-network policy [16, 1, 8, 9] — while retaining the full NP-hardness and non-uniform visibility structure of the problem. The public instance library provides *point-guard* solutions (guards anywhere in s); these are not the vertex-guard optimum and are not used as OPT here (see Section 5.1).

2.3 Evaluation metrics

Let N denote the cardinality of the evaluation partition under discussion. Mean coverage $\frac{1}{N} \sum \text{Cov}(S)$ summarizes overall performance, but the per-polygon distribution behind the mean is at least as informative: we report the counts $\#\{\text{Cov}(S) \geq c\}$ for threshold values $c \in \{0.95, 0.99, 0.999, 1\}$, and the worst-case polygon coverage $\min_s \text{Cov}(S)$. Note that c is distinct from the training-time coverage gate $\tau = 0.99$ used to label local-search targets (Section 4.4). Solution compactness is measured by two ratios: the normalized guard count $|S|/n$, which is the guard set size relative to the polygon’s vertex count, and the approximation ratio $|S|/\text{OPT}$, relative to the vertex-guard optimum. For binomial count proportions we report Wilson 95% confidence intervals on $\#\{\text{Cov}(S) \geq c\}/N$.

2.4 Geo-free inference

We will say that an inference procedure is *geo-free* if its inputs at test time are limited to the polygon’s vertex coordinates and learned features derived from them — no polygon visibility matrix, no CGAL-computed visibility regions, no per-vertex area or visibility-fraction feature. Geo-free inference does not preclude using exact visibility during evaluation (to report coverage after the fact) or during training (to compute rewards, gate trajectories, or build supervised targets); it restricts what the model reads at the moment it places guards. A solver allowed to query a visibility oracle at every decision step can in principle simulate greedy or local search, and a measurement of what such a solver has learned would be conflated with a measurement of what its oracle can compute. The geo-free constraint isolates the first question from the second, which is what makes “no explicit geometric knowledge” a measurable property and not just a rhetorical one.

3 Related Work

This section places the present work in three lines of research: classical algorithms for the Art Gallery Problem, neural combinatorial optimization with reinforcement learning, and supervised post-processing of learned solvers.

The Art Gallery Problem. Lee and Lin [10] established NP-hardness for several variants of the AGP; O’Rourke [12] surveys the foundational geometric and combinatorial results. The vertex-guard variant has a long history of approximation algorithms and exact integer-programming approaches; we treat published instance libraries [5] as the source of polygons in our experiments. Classical greedy heuristics [7] place guards to maximize marginal coverage and are reliable on small to mid-sized polygons; local search refines candidate guard sets by swapping or removing vertices and is the standard high-quality baseline. We use these classical methods only as evaluation reference points and as a source of supervised labels for the representation probe in Section 4.3; they are not in competition with the reinforcement method on guard count.

Reinforcement learning for combinatorial optimization. Pointer networks [16] introduced the architecture of attending over input tokens to produce variable-length output index sequences, and were extended to combinatorial optimization with reinforcement learning by Bello et al. [1] using REINFORCE [17] with a learned baseline. Subsequent work introduced attention-only architectures for routing problems [8] and policy optimization variants such as POMO [9] that exploit problem symmetries. We adopt the more recent preference-optimization (PO) recipe of Pan et al. [13], which trains the policy with a Bradley–Terry [4] ranking loss over pairs of solutions sampled from the policy itself, replacing the variance-heavy REINFORCE objective. We treat PO as a reinforcement-style procedure: the loss is computed from policy rollouts on

the environment (the polygon), the gradient direction is determined by the reward (coverage versus guard usage), and no supervised labels for the action sequence are used. A broader methodological survey is provided by Bengio et al. [2].

Supervised post-processing of learned solvers. An orthogonal line of work post-processes a base solver’s output with a second model trained by supervised or imitation learning. For combinatorial problems with hard feasibility constraints (such as covering), iterative supervised editors must contend with compounding errors and a train–inference distribution gap; DAgger [14] and its variants partially address the latter. We use a single-shot supervised post-processor (the SetPredictor) not as a competing solver but as a *probe* of the RL-trained encoder’s representation. The framing is methodological: if a small classifier reading frozen RL embeddings can recover feasibility, the RL encoder must already carry the relevant geometric information.

4 Method

We describe the reinforcement method (Section 4.1), the diagnostic that explains why a learned *iterative* editor over the same policy does not improve over it (Section 4.2), and the single-shot representation probe (Sections 4.3–4.5).

4.1 The reinforcement method: PO/BT pointer policy

Architecture. The policy is an LSTM-encoder pointer network with attention-based decoder, of roughly 364k parameters. A polygon s with vertex sequence $V(s) = (x_1, \dots, x_n)$ is consumed coordinate-wise (no visibility input). The encoder produces per-vertex embeddings $\{h_i\}_{i=1}^n$, $h_i \in \mathbb{R}^{128}$. The decoder attends over the encoder at each step, emits a vertex index or an EOS token, and stops at EOS or when the maximum length is reached. The decoder outputs a sequence $\pi = (\pi_1, \dots, \pi_T)$; the resulting guard set is $S(\pi) = \{\pi_t : \pi_t \neq \text{EOS}\}$, and all metrics (coverage, $|S|/n$, $|S|/\text{OPT}$) are computed on $S(\pi)$.

Reward and rollouts. For each polygon s , we sample R rollouts from the policy and score each rollout by a scalar utility $u(\pi | s)$ that combines coverage and guard usage; the precise scalarization places coverage above a soft threshold and then prefers smaller guard sets. Coverage during training is computed using a discretized-visibility sampler ($M=500$ interior points per polygon) as a cheap approximation; CGAL exact visibility is reserved for evaluation.

Preference-optimization loss. For each pair (π^+, π^-) of rollouts with $u(\pi^+) > u(\pi^-)$ above a coverage gate, we apply the Bradley–Terry log-likelihood

$$\mathcal{L}_{\text{BT}}(\theta) = -\mathbb{E}_{(s, \pi^+, \pi^-)} \left[\log \sigma(\alpha [\log p_\theta(\pi^+ | s) - \log p_\theta(\pi^- | s)]) \right], \quad (3)$$

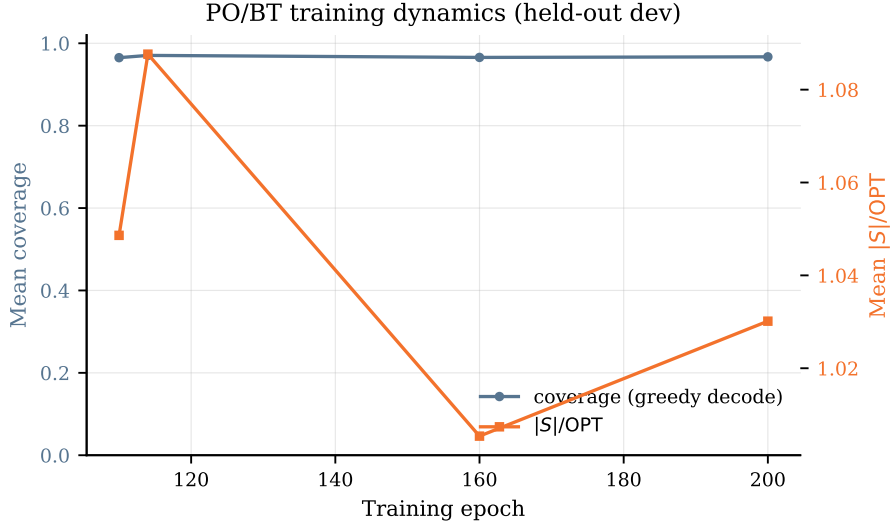


Figure 1: PO/BT training dynamics on the held-out `dev` split. Left axis: mean per-polygon coverage of the greedy-decoded policy over training epochs. Right axis: mean approximation ratio $|S|/OPT$ (or normalized guard count $|S|/n$ if `OPT` logging was disabled). The two curves together show that PO/BT improves coverage and guard cardinality jointly rather than trading one for the other, and that the preference-based gradient direction does not collapse as rewards saturate.

where $\log p_\theta$ is length-normalized over the index sequence (Pan et al. [13]’s length-control trick, since AGP solutions have variable length). The gradient direction is driven purely by which of the two rollouts achieved a higher reward; no supervised target sequence is used.

REINFORCE with a learned baseline is the classical choice for training pointer policies on combinatorial problems [1], but on the vertex-guard AGP we observed that rewards saturate quickly into a bimodal regime (covered versus uncovered) where REINFORCE advantages collapse and gradient information vanishes. PO sidesteps this: binary preferences over pairs of rollouts retain useful gradient direction even when the rewards themselves become similar, an empirical observation Pan et al. [13] highlight on routing problems and one we replicate here. At the end of training we are left with a single deterministic policy that maps a polygon’s coordinates to a guard sequence in one forward pass; this is the artifact whose properties the rest of the paper investigates, and from this point on its encoder is frozen and treated as a learned feature extractor. Figure 1 reports the held-out training dynamics and shows that the coverage–cost trade-off improves smoothly over the training run rather than collapsing into a degenerate regime.

4.2 What the reinforcement method does and does not absorb

The reinforcement method ends up absorbing two qualitatively different parts of the problem in qualitatively different ways. On the cardinality axis it is broadly successful: on held-out in-distribution polygons (Section 6.1) the policy reaches mean $|S|/\text{OPT} \approx 1.09$, which is competitive with classical heuristics on these sizes, and it does so without ever being shown a visibility object. On the feasibility axis it is less successful: on the same evaluation, 19.3% of polygons end up below 0.95 coverage with a worst case of 0.830, and on out-of-distribution polygons the coverage tail blows up to 14.7% of polygons below 0.95 with a worst case of 0.049. The reinforcement method has learned to use a small number of vertices, but it has not learned to guarantee that those vertices cover the polygon.

The natural reaction to this is to train a second learned model that applies a sequence of ADD/REMOVE/STOP edits to the policy’s seed. We tried three such iterative editor architectures of increasing capacity — including one with topology features and an auxiliary visibility-prediction loss, and one with DAGger refresh [14] — and none of them improved over the pretrained policy in any coverage-preserving regime. Three structural reasons make iteration difficult here: rollouts accumulate errors because the editor sees clean trajectories at training but its own evolving predictions at inference; a single classification head over $\{\text{ADD}, \text{REMOVE}, \text{STOP}\} \times \{1, \dots, n\}$ couples action selection with termination, producing STOP probabilities that are either too eager or too reluctant; and the training-time and inference-time state distributions diverge in ways that DAGger refresh alone did not close. We mention this only to motivate the single-shot design used for the representation probe: there is no rollout, there is no STOP token, and the model is trained and used in the same regime.

4.3 Probing the representation: a single-shot per-vertex classifier

The hypothesis under test asks what the reinforcement method has internalized. A natural way to test this is to ask whether a small classifier, conditioned only on the encoder’s output and never on a visibility object, can recover the feasibility decision the decoder failed to make. We call this classifier the *SetPredictor*.

For polygon s with vertex sequence $V(s) = (x_1, \dots, x_n)$, the input feature for vertex i is

$$z_i = [h_i; x_i; \mathbf{1}\{i \in \pi_{\text{seed}}\}] \in \mathbb{R}^{131}, \quad (4)$$

that is, the *frozen* 128-dimensional encoder embedding h_i from the PO/BT policy, the 2D coordinates x_i , and a single binary indicator for whether vertex i is selected by the policy’s greedy-decoded seed π_{seed} . No visibility matrix, no CGAL output, and no area feature enters the classifier at inference time. The geo-free constraint is preserved.

The architecture is a small Transformer encoder over the n vertex tokens: 3 self-attention layers, 8 heads, hidden width 128, FFN expansion factor 4, and

a sigmoid output head per token. Total parameters: $\sim 464\text{k}$. We form a per-vertex representation by concatenating the attention output with two pooled context vectors — mean-pooled over the seed set and mean-pooled over all valid (non-padded) vertices — and feed the concatenation to a 2-layer MLP head:

$$\hat{p}_i = \sigma(\text{MLP}([v_i \parallel S_{\text{pool}} \parallel G_{\text{pool}}])_i), \quad i = 1, \dots, n. \quad (5)$$

The output $\hat{p}_i \in [0, 1]$ is interpreted as the probability that vertex i should be in the final guard set. Thresholding at $t \in (0, 1)$ yields the guard set

$$S(t) = \{i \in \{1, \dots, n\} : \hat{p}_i \geq t\}. \quad (6)$$

The SetPredictor is trained by supervised learning on labels derived from local search over the policy’s seed (Section 4.4), so it is not itself a reinforcement learner; we use it as a probe of the RL-trained encoder. The only polygon-structural information it reads at inference is the frozen RL embedding h_i together with x_i and the seed indicator, and the logic of the probe is direct: if a small classifier reading only those embeddings can close the feasibility tail, the encoder must already carry the geometric information needed to support that decision.

4.4 Training the probe

Target sets. For each training polygon s we obtain a target set $\pi_{\text{LS}}^*(s)$ by applying local search to the policy’s seed solution. The local search uses a coverage gate $\tau = 0.99$ relative to a reference computed with disc-vis sampling ($M = 500$), which is the cheap approximation we use at training time. CGAL exact visibility is used only at evaluation.

Loss. Binary cross-entropy per vertex with a positive-class weight that automatically balances the empirical guard/non-guard ratio (mean ≈ 5.66 on the development split),

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{B}|} \sum_{s \in \mathcal{B}} \frac{1}{|V(s)|} \sum_{i=1}^{|V(s)|} [w_+ y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)], \quad (7)$$

where $y_i = \mathbf{1}\{i \in \pi_{\text{LS}}^*(s)\}$ and w_+ is the positive-class weight.

Hyperparameters. 60 epochs, batch size 32, learning rate 3×10^{-4} , AdamW [11] with default weight decay, single random seed (1234). Checkpoint selection is on the `dev_tune` split (Section 5).

4.5 Inference and the threshold knob

At inference time the procedure is: run the PO/BT policy’s encoder on the polygon’s coordinates (one forward pass), greedy-decode a seed π_{seed} , form the

per-vertex features z_i of Eq. (4), run one forward pass of the SetPredictor to obtain $\{\hat{p}_i\}_{i=1}^n$, and return $S(t)$ from Eq. (6). No CGAL call, no visibility matrix, no oracle anywhere on this path. The threshold t is the single continuous knob that controls the coverage-size trade-off: lower t admits more vertices, raising both expected coverage and $|S|$, and we report the full Pareto curve over $t \in \{0.20, 0.25, 0.30\}$ in Section 6.3. One could in principle iterate the SetPredictor by re-running it with the previous prediction’s in-set indicator updated, for K passes; we evaluated $K \in \{1, 2, 3, 5\}$ over a sweep of thresholds and found that the predictions are empirically a fixed point after the first pass (Section 6.4). This is the expected behavior of a single-shot model whose input already includes the seed indicator, so we use $K = 1$ throughout.

5 Experiments

5.1 Dataset and partition

We draw polygons from the publicly available AGP instance library of Couto, de Rezende, and de Souza [5], which contains four families (random simple, random orthogonal, random von Koch, complete von Koch) over a wide range of vertex counts n . The library distributes *point-guard* optimal solutions — guards may be placed at any point of the polygon, not only at vertices — which are not the optimum for the vertex-guard variant we study. We therefore compute vertex-guard optima OPT ourselves, by solving the vertex-guard set-cover integer program for each polygon, and use these computed OPT values throughout the paper. The splits we work with are listed in Table 1.

The 70/30 partition is critical: an earlier alphabetical-name split caused systematic family imbalance between `dev_tune` and `dev_test` (`fat-*` polygons concentrated in tune, `randsimple-*` in test), which produced misleading optimism. The seeded random shuffle removes this bias.

5.2 Training setup

The PO/BT reinforcement-trained policy is trained once and frozen; all coverage rewards during PO training use disc-vis sampling. The SetPredictor probe is trained on `train` only; `dev_tune` is used for early stopping and threshold inspection; `dev_test`, `test`, and `large` are touched only after training and threshold selection are complete. Hyperparameters for the SetPredictor are stated in Section 4.4; the policy itself is the existing `lstm_bt` checkpoint produced by our PO/BT training script.

5.3 Evaluation protocol

For each polygon we compute the policy’s seed (the policy’s own guard sequence, EOS-truncated and greedy-decoded), evaluate it with CGAL exact polygon visibility [15], and then, separately, run a single SetPredictor forward pass on the policy’s encoder output, threshold at t to obtain $S(t)$, and evaluate $S(t)$ with

Split	# polygons	n range	Use
train	9,791	8–150	Pointer + SetPredictor training
dev_tune	857	8–150	SetPredictor checkpoint selection
dev_test	367	8–150	Held-out evaluation (in-distribution)
test	2,107	12–1,000	Out-of-distribution evaluation
large	20	1,000–2,250	Extreme OOD (smoke)

Table 1: Dataset partition. The **dev** split is partitioned 70/30 into **dev_tune** (checkpoint selection) and **dev_test** (held-out evaluation) by seeded random shuffle (seed 1234). The **test** split contains polygons whose vertex count exceeds the training range and is used as the out-of-distribution evaluation. The **large** split is reserved for an extreme-OOD smoke evaluation.

CGAL. We report the metrics defined in Section 2.3: mean coverage $\text{Cov}(S)$, mean normalized size $|S|/n$, mean approximation ratio $|S|/\text{OPT}$, and the per-polygon coverage counts $\#\{\text{Cov}(S) \geq c\}$ for $c \in \{0.95, 0.99, 0.999, 1\}$, plus the minimum coverage. Wilson 95% confidence intervals are reported on the proportion $\#\{\text{Cov}(S) \geq 0.95\}/N$.

As noted in Section 5.1, vertex-guard optima OPT are not provided by the public library (which distributes point-guard solutions) and are computed by us via integer programming. Our OPT values are available for all polygons in **train**, **dev**, and **test**, and unavailable for some polygons in **large** (where the ILP was too slow to terminate within our compute budget); we report coverage only on those.

6 Results

We organize the results around the hypothesis. Section 6.1 reports what the reinforcement method achieves on its own — the answer to “does the policy learn to place guards.” Section 6.2 examines the per-polygon distribution behind the mean. Sections 6.3–6.4 report the representation probe’s Pareto curve and its fixed-point property. Section 6.5 is the main out-of-distribution measurement, which is the most informative test of whether the reinforcement-learned representation generalises.

6.1 The reinforcement method places guards

The first row of Table 2 is the geo-free reinforcement policy on its own: mean coverage 0.969, 296/367 polygons with $\text{Cov}(S) \geq 0.95$ (Wilson 95% CI (0.763, 0.844)), and $|S|/\text{OPT} \approx 1.09$. Read this against the question we asked in the introduction. A policy that sees only polygon coordinates and is trained from coverage-aware rewards over its own rollouts produces, on a held-out partition, guard sets that are competitive in cardinality and reach 0.95 coverage on the large majority of polygons. The hypothesis — that vertex-guard placement is learnable, to a

Method	Mean coverage	$\#\{\text{Cov} \geq 0.95\}/N$	Mean $ S /\text{OPT}$
Pretrained pointer (seed)	0.9689	296/367 (0.763,0.844)	1.0878
SetPredictor ($t = 0.20$)	0.9993	367/367 (0.990,1.000)	2.9007

Table 2: Held-out evaluation on `dev_test` (367 polygons). *Pretrained pointer (seed)* is the PO/BT-trained policy decoded greedily, the artifact whose properties this paper investigates. The SetPredictor row reports the representation probe at $t = 0.20$. Wilson 95% CI on the coverage-feasibility proportion is shown in parentheses.

Method	$\#\{\text{Cov} = 1\}$	$\#\{\text{Cov} \geq .999\}$	$\#\{\text{Cov} \geq .99\}$	$\#\{\text{Cov} \geq .95\}$	min Cov
Pretrained pointer (seed)	3	15	99	296/367	0.830
SetPredictor $t = 0.20$	243	315	363	367/367	0.977
SetPredictor $t = 0.25$	195	283	361	367/367	0.962
SetPredictor $t = 0.30$	152	221	355	367/367	0.952

Table 3: Per-polygon coverage distribution on `dev_test` (367 polygons). Counts show the number of polygons reaching each coverage threshold; $\min \text{Cov}(S)$ is the worst polygon. The pretrained policy has a long left tail. The SetPredictor moves the entire distribution to the right.

degree, by a reinforcement method that never sees an explicit visibility object — is confirmed in the affirmative on this measurement.

The honest scope of the confirmation is the residual: $71/367 = 19.3\%$ of polygons fall below $\text{Cov}(S) = 0.95$ under the policy alone, and the minimum coverage is 0.830. The reinforcement method, on its own, does not give a feasibility guarantee. Figure 2 draws two illustrative polygons, one from `dev_test` and one from `test`, with the policy’s seed, the policy plus the SetPredictor probe at $t = 0.30$, and the optimum drawn side by side; it gives a visual sense of where the policy succeeds and where the probe contributes.

6.2 Per-polygon distribution behind the mean

The mean coverage 0.969 of the policy alone hides a bimodal distribution. Table 3 reports the per-polygon counts. The policy reaches $\text{Cov}(S) = 1$ on only 3/367 polygons, $\text{Cov}(S) \geq 0.99$ on 99/367, and has 71 polygons below 0.95 with a worst case of 0.830. The probe, at $t = 0.20$, eliminates the left tail and moves the bulk of the distribution above 0.99. Figure 3 visualizes the same shift as a per-polygon complementary CDF, with the dashed line at 0.95 marking the feasibility gate; the SetPredictor curve sits to the right of the pretrained policy at every coverage threshold, in both the in-distribution and the out-of-distribution panel.

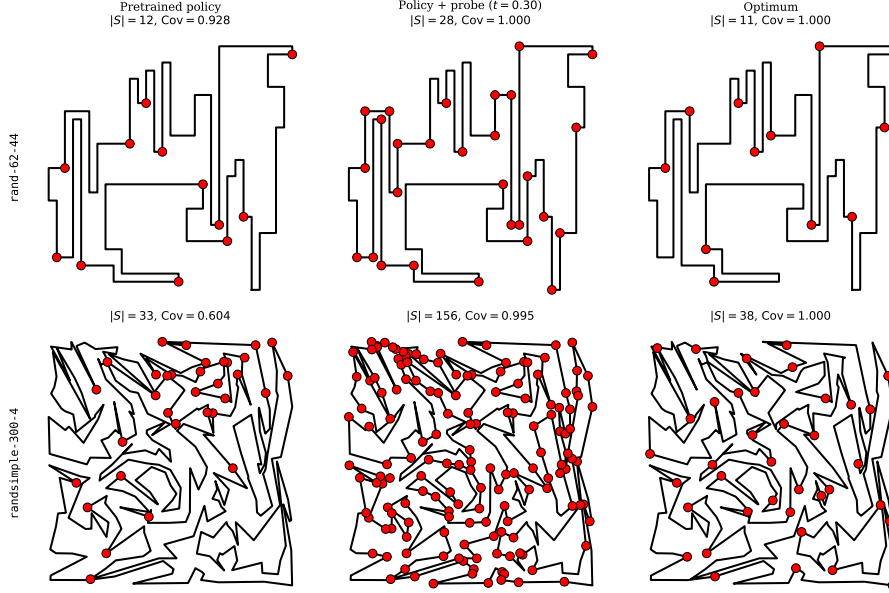


Figure 2: Worked examples: two polygons drawn with the policy’s seed (left column), the policy plus the SetPredictor probe at $t = 0.30$ (middle column), and the optimum (right column). Vertices selected as guards are marked. Row 1 is an in-distribution polygon where the policy already covers most of the region. Row 2 is an out-of-distribution polygon where the policy’s seed leaves a substantial uncovered region and the probe restores feasibility by adding vertices the policy missed.

6.3 The representation probe: Pareto curve

Table 4 traces the Pareto curve from $t = 0.30$ to $t = 0.20$. All three thresholds satisfy the coverage gate $\text{Cov}(S) \geq 0.95$ on every `dev.test` polygon; $t = 0.20$ is the most conservative (largest $|S|$, fewest residual sub-0.99 polygons). Between $t = 0.30$ and $t = 0.20$, mean coverage rises by ≈ 0.0015 and mean $|S|/n$ rises by ≈ 0.10 . The corresponding cost in guard count, measured against the optimum, rises from ≈ 1.09 for the reinforcement policy alone to ≈ 2.20 at $t = 0.30$ and ≈ 2.90 at $t = 0.20$. This is the cost of feasibility: the probe trades guard count for the missing coverage. Figure 4 plots the same trade-off across a broader threshold sweep on `dev.tune`, together with the three canonical operating points on `dev.test` and on the OOD `test` split.

6.4 Iterative inference of the probe is a fixed point

Table 5 confirms that re-running the probe with its own output as the new in-set indicator does not change the prediction: across $K \in \{1, 2, 3, 5\}$ the changes

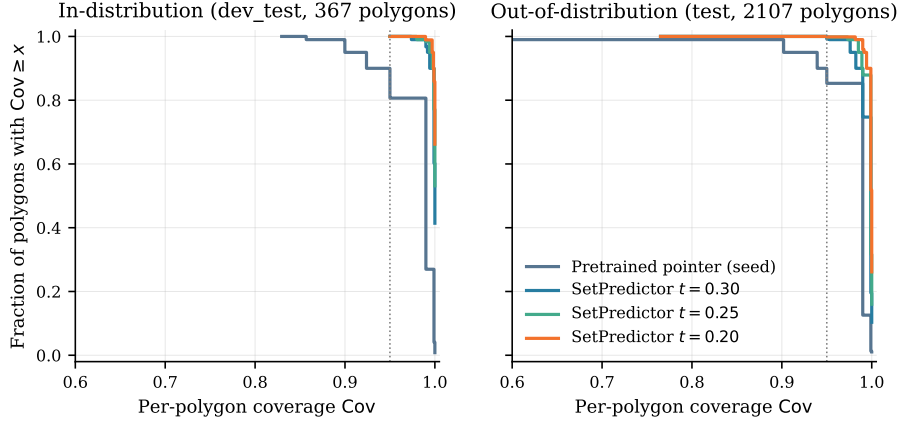


Figure 3: Per-polygon coverage distribution as a complementary stair-step CDF, in-distribution `dev_test` (left, 367 polygons) and out-of-distribution `test` (right, 2107 polygons). For each method we plot the fraction of polygons with coverage at least x against x . The dashed vertical line marks the 0.95 feasibility gate. Across both panels and across all three SetPredictor thresholds, the SetPredictor curve sits to the right of the pretrained policy.

Threshold t	Mean Cov	Mean $ S /n$	Mean $ S /\text{OPT}$	$\#\{\text{Cov} \geq .95\}/N$
0.20	0.9993	0.4376	2.9007	367/367
0.25	0.9988	0.3802	2.5091	367/367
0.30	0.9979	0.3352	2.2030	367/367

Table 4: Threshold sweep on `dev_test` (367 polygons). Lower t admits more vertices, raising both expected coverage and $|S|$. All three operating points reach $\#\{\text{Cov}(S) \geq 0.95\}/N = 367/367$.

in mean coverage, $|S|/n$, and $|S|/\text{OPT}$ are at the fourth decimal place. The probe is a fixed point of its own prediction. Figure 5 plots the same information across six thresholds; the lines are visibly flat against the small vertical range, confirming that one forward pass is sufficient. We use $K = 1$ in the rest of the paper.

6.5 Out-of-distribution evaluation

Table 6 is the strongest available test of what the reinforcement-trained representation generalizes. On polygons whose vertex count exceeds the policy’s training range, the policy alone falls back to mean coverage 0.957 with 310/2107 polygons below 0.95 and a worst case of 0.049. The same probe checkpoint, trained on the in-distribution range, reduces sub-0.95 failures by approximately two orders of magnitude — from 310 to 3 of 2107 at $t = 0.20$ — and lifts the

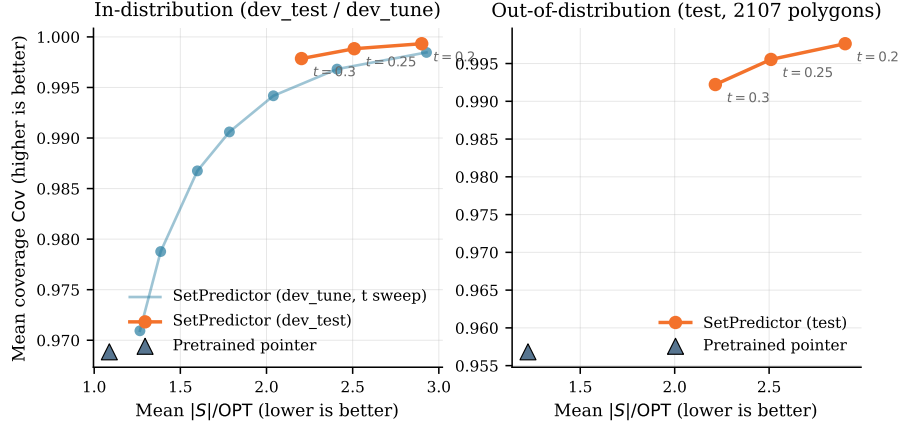


Figure 4: Coverage–cost trade-off, in-distribution (left) and out-of-distribution (right). The horizontal axis is the mean guard count relative to the optimum, $|S|/\text{OPT}$; the vertical axis is mean per-polygon coverage. The pretrained policy (triangle) sits at low cost and the bottom of its coverage curve; the SetPredictor (circles) traces a monotone curve as the threshold t is lowered, gaining coverage at a controlled cost in $|S|/\text{OPT}$. The left panel includes the broader **dev_tune** threshold sweep as a faint connecting curve.

worst polygon’s coverage from 0.049 to 0.765.

The reading we propose for this result is twofold. First, it confirms that the reinforcement-learned encoder generalizes beyond the size range it was trained on, at least in terms of carrying the information a small classifier needs to recover feasibility. Second, it bounds how much: even at the lowest threshold we report, the worst polygon’s coverage is 0.765, not 1. Generalization is partial. Figure 6 disaggregates this by polygon size: per-polygon coverage as a function of vertex count n on the OOD split, with the policy and the probe shown side by side.

Inference passes K	Mean Cov	Mean $ S /n$	Mean $ S /\text{OPT}$
$K = 1$	0.9723	0.1968	1.2328
$K = 2$	0.9712	0.1966	1.2302
$K = 3$	0.9719	0.1966	1.2295
$K = 5$	0.9719	0.1966	1.2292

Table 5: Inference passes $K \in \{1, 2, 3, 5\}$ at fixed threshold ($t = 0.65$, full dev, 1224 polygons). The variation across K is at the fourth decimal: iterating the SetPredictor returns essentially the same prediction.

Method	Mean Cov	$\#\{\text{Cov} \geq .95\}/N$	min Cov	Mean $ S /\text{OPT}$
Pretrained pointer (seed)	0.9568	1797/2107	0.049	1.2233
SetPredictor $t = 0.20$	0.9976	2104/2107	0.765	2.9026
SetPredictor $t = 0.25$	0.9955	2103/2107	0.765	2.5098
SetPredictor $t = 0.30$	0.9922	2090/2107	0.765	2.2153

Table 6: Out-of-distribution evaluation on the **test** split (2107 polygons, vertex counts up to $n = 1000$). The reinforcement policy’s coverage tail blows up on out-of-distribution polygons (310 of 2107 below $\text{Cov}(S) = 0.95$, with a minimum coverage of 0.049); the probe at $t = 0.20$ reduces $\#\{\text{Cov}(S) < 0.95\}$ to 3, a $\sim 100\times$ reduction.

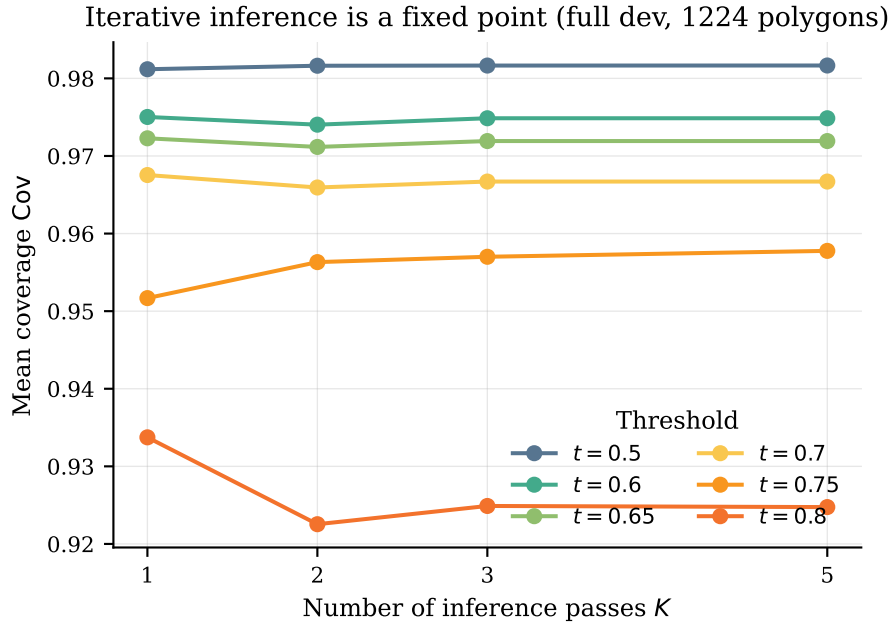


Figure 5: Iterative inference is empirically a fixed point. Mean coverage as a function of the number of inference passes $K \in \{1, 2, 3, 5\}$, for six thresholds $t \in \{0.5, 0.6, 0.65, 0.7, 0.75, 0.8\}$, on the full dev split (1224 polygons). The vertical axis is tightly cropped to show that the across- K variation is at the fourth decimal at every threshold.

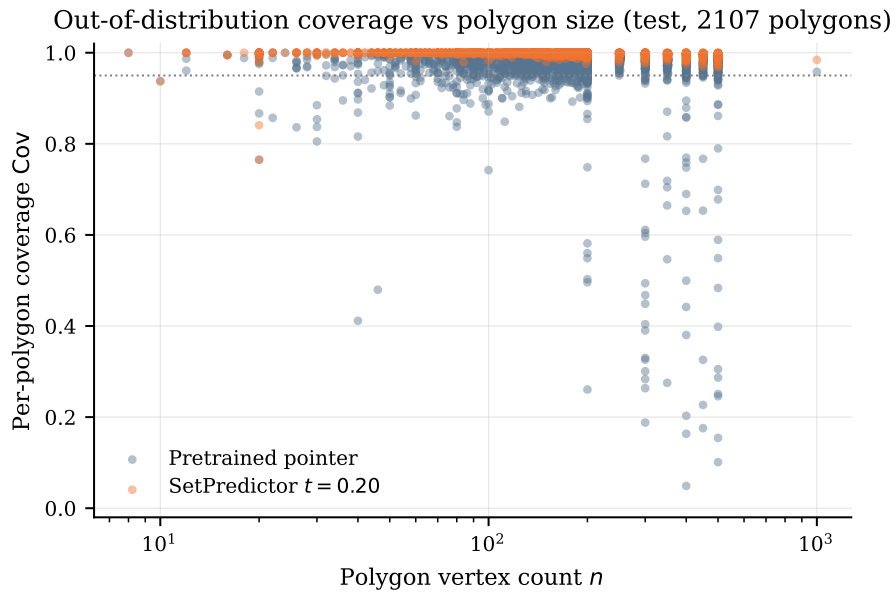


Figure 6: Per-polygon coverage as a function of polygon vertex count n on the OOD **test** split ($n \in [12, 1000]$). The pretrained policy’s coverage degrades systematically as n grows; the SetPredictor probe at $t = 0.20$ stays near the 0.95 feasibility line across the entire size range. This is the size-resolved view of the OOD generalization story summarized by Table 6.

7 Discussion

Read against the question the introduction posed, the reinforcement method does learn to place guards on the polygon, and it does so without ever seeing an explicit visibility object. The geo-free policy reaches mean coverage of roughly 0.969 at $|S|/\text{OPT} \approx 1.09$ on the held-out in-distribution split, with 296/367 polygons (Wilson 95% CI (0.763, 0.844)) clearing the 0.95 coverage gate. The cardinality of the guard sets is competitive and the bulk of polygons end up acceptably covered; “competitive,” not “optimal,” is what the question called for.

The honest counterpart to that finding is the residual tail. Roughly 19.3% of in-distribution polygons and 14.7% of out-of-distribution polygons fall below 0.95 coverage under the policy alone, and these are not minor near-misses: minimum coverages drop to 0.830 in-distribution and 0.049 out-of-distribution. The reinforcement method, by itself, is not a feasibility-guaranteeing procedure, and we read this as the limit of what pure RL delivers in our setup at this scale.

The single most informative measurement in the paper is what happens when we read the encoder rather than the decoder. The SetPredictor probe sees only frozen encoder embeddings and the policy’s own seed indicator — no visibility matrix, no CGAL output, no per-vertex area feature — and it closes the in-distribution feasibility tail completely (367/367 polygons above 0.95 at $t = 0.20$) and the out-of-distribution tail to within 3 polygons of 2107 at the same threshold. There are two reasonable interpretations of this result. One is that the encoder learned enough geometric structure that a small classifier reading its output can recover feasibility on its own; the other is that the decoder’s EOS calibration was the bottleneck and the policy had already identified the right vertices but stopped too early. Both interpretations support the same reading of what the reinforcement method internalized: a representation that already contains more of the relevant geometry than the policy’s decoder was able to express. The cost of recovering feasibility through the probe is a controlled rise in guard count, from $|S|/\text{OPT} \approx 1.09$ under the policy to roughly 2.20 at $t = 0.30$ and 2.90 at $t = 0.20$, and we report it explicitly rather than disguise it. Figure 7 makes the representation argument visual: the encoder’s per-vertex embeddings, projected to two dimensions and colored by guard-membership label, separate without ever having been shown a visibility matrix.

It is worth restating what this paper is not claiming. Classical greedy and local-search heuristics outperform the policy on $|S|/\text{OPT}$ at preserved coverage, and we have not designed the work to contest this; the policy and the probe are not presented as competitors to those methods on guard count. What we are documenting is what is learnable by reinforcement when the model is denied geometric input at the moment of placing guards — and, jointly, what part of that learning lives in the representation rather than in the decoder. The two formulations of post-hoc improvement we tried bear on the same point. An iterative editor trained over the policy’s seed could not improve coverage in any preserved-cardinality regime, and the single-shot probe is empirically a fixed point under iteration; what changes between the two is whether a model

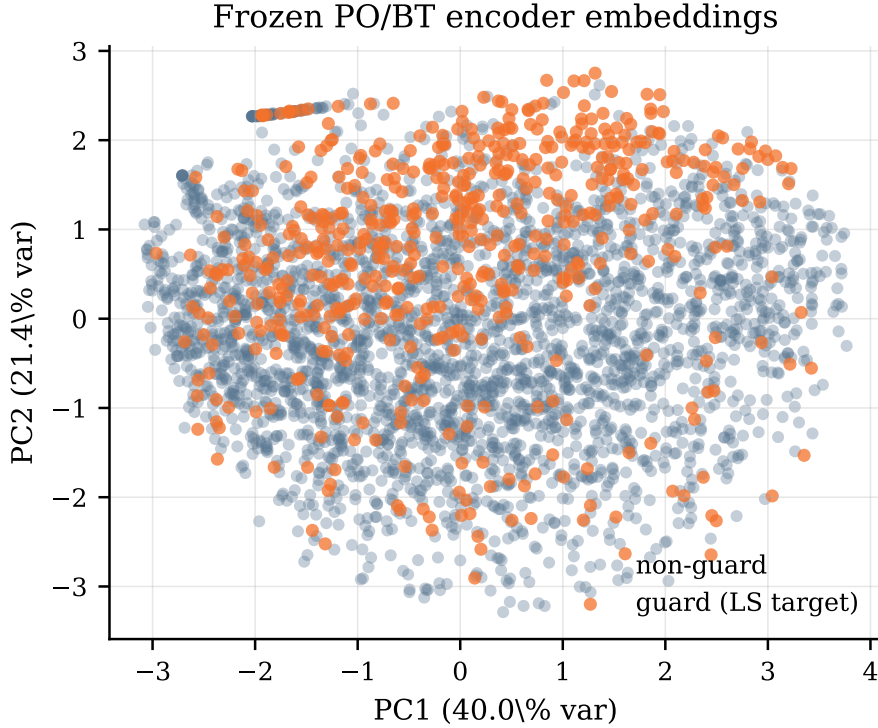


Figure 7: Two-dimensional projection of the frozen PO/BT encoder’s per-vertex embeddings, colored by guard-membership label (LS-derived target). Each point is one vertex of one polygon, pooled across a sample of held-out polygons. Visible separation between guard and non-guard vertices, in a projection that has no access to the visibility matrix, is direct visual evidence that the reinforcement-trained encoder has internalized a guard-relevant geometric signal.

is asked to choose a stop time and a sequence of edits or simply to produce a probability per vertex. The single-shot formulation is the one that exposes the encoder’s information, and it does so precisely because it removes everything else.

8 Limitations

1. **Scope of the “no geometric knowledge” claim.** The geo-free constraint is on inference: the policy and the probe see no visibility object at test time. *Training* does see visibility: the PO/BT reward is computed from disc-vis coverage, and the probe’s supervised targets are generated

by local search using disc-vis as a coverage gate. “No explicit geometric knowledge” must be read as “no geometric oracle at the moment of placing guards.”

2. **The probe is supervised, not reinforcement.** The SetPredictor is trained by BCE against local-search-derived labels. It is not itself a reinforcement learner. We position it as a representation probe of the RL-trained encoder, not as an RL contribution.
3. **Held-out partition size.** `dev_test` has 367 polygons. Wilson 95% CIs on the coverage-feasibility proportion are reported, but the absolute count is modest. The OOD evaluation on `test` (2107 polygons) provides a much larger second held-out evidence base.
4. **Single random seed.** Both the policy and the probe are trained with a single seed (1234). Multi-seed runs and seed-variance confidence intervals are left for future work.
5. **LSTM, not Transformer, encoder.** The reinforcement-trained policy is an LSTM pointer. A Transformer encoder might produce stronger embeddings and tighten the Pareto curve; we did not retrain.
6. **disc-vis approximation in training.** Both the PO/BT coverage reward and the SetPredictor training targets use disc-vis sampling at $M = 500$. Exact CGAL visibility is used only at evaluation.
7. **Extreme OOD unaddressed in the main results.** The `large` split (n up to 2250) is not evaluated in the main results table. A smoke run on 20 polygons confirms the probe still tracks the threshold curve; a full evaluation requires further engineering.

9 Conclusion

We have asked whether a neural policy can learn to place vertex guards on a simple polygon from a coverage-aware reward over its own rollouts, with no geometric oracle at the moment it chooses guards, and we have answered the question to a measurable degree. The LSTM pointer-network policy trained by preference optimization on Bradley–Terry pairs reaches mean coverage of roughly 0.969 at $|S|/\text{OPT} \approx 1.09$ on held-out polygons; a non-trivial fraction of polygons remain under-covered, and the same effect is more pronounced on out-of-distribution polygons whose vertex counts exceed the training range. Reading the encoder rather than the decoder, through a small per-vertex classifier that sees only frozen embeddings and the seed indicator, closes the feasibility tail without any geometric input at inference, and we take this as direct evidence that the reinforcement-trained representation already carries more of the geometry than the policy’s own decoder revealed. The cost of recovering feasibility through that probe is a controlled rise in guard count.

We position this work as a measurement of what is learnable in a deliberately constrained setting, not as a competitor to classical solvers on guard count. The reinforcement learner is not the best AGP heuristic; it is a learner that arrived at a usable representation of vertex-guard placement without ever being shown the geometry it had to reason about. Two natural extensions follow. A stronger encoder — for example, a Transformer trained under the same PO/BT objective — may shorten the residual coverage tail directly, leaving less work for the probe. A per-instance threshold predicted from the same encoder embeddings would replace the global threshold with an adaptive one and may further close the gap between feasibility and cardinality.

References

- [1] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*, 2016.
- [2] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.
- [3] Bahram Sadeghi Bigham, S. Dolatikalan, and Alireza Khastan. Minimum landmarks for robot localization in orthogonal environments. *Evol. Intell.*, 15(3):2235–2238, 2022.
- [4] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3–4):324–345, 1952.
- [5] Marcelo C. Couto, Pedro J. de Rezende, and Cid C. de Souza. Instances for the Art Gallery Problem. <http://www.ic.unicamp.br/~cid/Problem-instances/Art-Gallery>, 2009.
- [6] A. Elnagar and L. Lulu. An art gallery-based approach to autonomous robot motion planning in global environments. In *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 2079–2084, 2005.
- [7] Héctor González-Baños. A randomized art-gallery algorithm for sensor placement. *Proceedings of the 17th Annual Symposium on Computational Geometry (SoCG)*, pages 232–240, 2001.
- [8] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations (ICLR)*, 2019.
- [9] Yeong-Dae Kwon, Jinho Choo, Byoungjip Kim, Iljoo Yoon, Youngjune Gwon, and Seungjai Min. POMO: Policy optimization with multiple optima

- for reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.
- [10] D. T. Lee and A. K. Lin. Computational complexity of art gallery problems. *IEEE Transactions on Information Theory*, 32(2):276–282, 1986.
 - [11] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
 - [12] Joseph O’Rourke. *Art Gallery Theorems and Algorithms*. Oxford University Press, 1987.
 - [13] Yu Pan et al. Preference optimization for neural combinatorial optimization. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
 - [14] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 627–635, 2011.
 - [15] The CGAL Project. CGAL user and reference manual. <https://doc.cgal.org/5.6/Manual/packages.html>, 2024.
 - [16] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.
 - [17] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992.